

Machine Learning Final Assignment

By Gaurav Naudiyal (Student ID: 25339213)

Msc Computer Science – Augmented & Virtual Reality

1.1 Dataset Design

To understand how the model handles arithmetic reasoning, I created multiple arithmetic datasets using code, starting simple and gradually increasing the difficulty. All expressions were written at the character level in the format:

expression=result

There were no spaces, and each expression appeared on its own line. This setup helps the model focus on individual digits and operators instead of treating numbers as hidden values.

Three dataset variants were used:

- **Single-digit addition:** expressions of the form $a + b = c$, where $a, b \in [0, 9]$
- **Two-digit addition:** expressions of the form $ab + cd = ef$, introducing carry propagation
- **Mixed addition and subtraction:** expressions combining $+$ and $-$, restricted to non-negative subtraction results

For single-digit addition, I tested both a random train–test split and a full dataset to see whether the model was memorising or actually generalising. For the other datasets, an 80/20 train–test split was used. The mixed addition and subtraction dataset was chosen as the final model because it best revealed where the model performs well and where it fails.

1.2 Evaluation Metrics

Arithmetic reasoning requires us to be very accurate; therefore, **exact-match accuracy** was used as the primary evaluation metric. A prediction was considered correct only if the generated output exactly matched the expected result after the equals sign.

Early experiments showed that probabilistic sampling during decoding significantly reduced accuracy, even when the model had learned correct mappings. Consequently, **greedy decoding** was used during evaluation. This choice reflects the deterministic nature of symbolic reasoning tasks and avoids artificially inflated error rates caused by any random generation.

1.3 Model Architecture and Adaptations

All of the arithmetic experiments were conducted using a small GPT-style transformer trained with next-token prediction and cross-entropy loss. A very minimal architecture was chosen by me so that I can be easier to expose biases and limitations rather than to maximise performance. That was my goal during this research.

Key design choices included:

- **Character-level tokenisation**, enabling explicit representation of digits and operators.
- **Small model size** (two Transformer layers, two attention heads, modest embedding dimension) to reduce overfitting.
- **Context length adaptation**: the maximum sequence length was increased for two-digit and mixed-operation datasets to accommodate longer expressions.

Unless otherwise stated, all Math GPT experiments used the same base architecture, with only the context length adjusted for more complex expressions.

1.4 Arithmetic Results and Analysis

Single-digit Addition

With the full single-digit dataset, the model performed almost perfectly, achieving around 99% accuracy. When a random train–test split was used and some outputs were missing from training, performance dropped to about 45%. The model mainly failed on expressions with unseen results, showing that it relied on memorising patterns rather than actually learning how addition works.

```
Exact-match accuracy: 0.990 (99/100)
```

```
Sample errors:
```

```
7+8= → predicted 14, expected 15
9+6= → predicted 14, expected 15
8+7= → predicted 14, expected 15
6+9= → predicted 14, expected 15
5+9= → predicted 13, expected 14
9+5= → predicted 13, expected 14
4+9= → predicted 12, expected 13
4+9= → predicted 12, expected 13
```

Two-digit Addition

On two-digit addition, the model achieved relatively high accuracy (approx. 86%). Despite this, systematic errors were observed, particularly around **carry propagation**, where local digit patterns were correct but global arithmetic consistency failed. These errors demonstrate that the

model captures strong statistical regularities without reliably implementing multi-step arithmetic rules.

```
Exact-match accuracy: 0.865 (346/400)
```

```
Sample errors:
```

```
82+32= → predicted 113, expected 114
```

```
22+66= → predicted 98, expected 88
```

```
67+33= → predicted 90, expected 100
```

```
23+80= → predicted 102, expected 103
```

```
10+64= → predicted 84, expected 74
```

Mixed Addition and Subtraction (Final Model)

Combining addition and subtraction resulted in a marked performance drop, with exact-match accuracy of approximately **53%**. Two dominant failure modes were observed:

1. **Carry and borrow propagation errors**, where intermediate arithmetic state was not maintained correctly.
2. **Partial operator confusion**, with some subtraction expressions producing addition-like outputs.

Representative failures include incorrect handling of carries (e.g. $70+35$) and borrow operations (e.g. $28-11$). While the model did learn to condition on operator symbols, it lacked strong operator-specific computation.

2.1 Dataset Design

To test symbolic reasoning without numbers, a Boolean logic dataset was created using code. Boolean logic is a good comparison to arithmetic because it uses a small, fixed set of symbols and does not involve things like size, carrying, or borrowing.

All expressions were represented at the **character level** using a consistent format:

```
expression=result
```

with no whitespace and one expression per line. Examples include:

```
TrueANDFalse=False  
NOTTrue=False
```

```
(TrueORFalse)ANDTrue=True  
TrueXORTrue=False
```

The dataset included the Boolean operators **AND**, **OR**, **NOT**, and **XOR**, as well as a small number of expressions containing parentheses to test limited compositional structure. Boolean values were restricted to `True` and `False`. The dataset was split into training and test sets using an 80/20 split.

2.2 Evaluation Metrics

Similar to the arithmetic experiments, we used exact-match accuracy to evaluate the model. An output was only marked correct if it matched the expected Boolean value exactly. Greedy decoding was used so the model always produced the same result. Random sampling was not used, since even small changes can lead to invalid Boolean answers.

2.3 Model Architecture and Adaptations

The Boolean GPT reused the same minimal GPT architecture employed for Math GPT. This design choice enables a direct comparison between arithmetic and Boolean reasoning under identical architectural constraints.

Key architectural features include:

- **Character-level tokenisation**, covering Boolean literals, operators, parentheses, and the equals sign.
- **Small model size** (two Transformer layers, two attention heads, modest embedding dimension).
- **Short context length**, sufficient for all Boolean expressions.

No additional architectural modifications were required for Boolean reasoning, as expression lengths were short and dependencies were local.

2.4 Boolean Logic Results and Analysis

Quantitative Results

The Boolean GPT learned fast, with the training loss dropping a lot in the first few hundred steps. The model reached moderate to high accuracy on the test set. Since the Boolean dataset was small, even a few mistakes had a big impact on the reported accuracy.

Qualitative Analysis

The model correctly evaluated a wide range of Boolean expressions, including both unary and binary operations:

```
TrueANDFalse → False
TrueORFalse → True
NOTTrue → False
```

Observed errors were limited in number and primarily involved **XOR** and, less frequently, **AND**:

```
!python model/evaluate.py
```

```
Exact-match accuracy: 0.500 (2/4)
```

```
Sample errors:
```

```
TrueXORTrue= → predicted True, expected False
```

```
TrueANDTrue= → predicted False, expected True
```

These mistakes show that the model sometimes mixed up operators, especially XOR. XOR is harder to learn compared to AND or OR. The errors followed clear patterns instead of being random, meaning the model still learned useful Boolean logic even with limited data.

3 Discussion

3.1 Architectural Effectiveness Across Tasks

One important finding is that the same small model worked well for Boolean logic but had trouble generalising in arithmetic tasks. Character-level tokenisation was suitable for both, since it let the model directly see digits and operators. Arithmetic expressions were longer and more complex, while Boolean expressions stayed short and simple.

Making the model bigger or increasing the context length did not solve the arithmetic issues. This shows that the model struggles with arithmetic because it learns patterns, not because it is too small.

3.2 Evaluation Strategy and Decoding

The way the model was evaluated and decoded made a big difference to its performance. Exact-match accuracy was needed because symbolic tasks do not allow partially correct answers. When random sampling was used, the model often gave wrong outputs even if it knew the correct result. Switching to greedy decoding showed what the model could actually do.

This shows a clear difference between language generation and symbolic reasoning. Variation is helpful in language tasks, but it causes problems when exact answers are required.

3.3 Failure Modes and Generalisation

The arithmetic tests showed several clear weaknesses. For single-digit addition, the model struggled with results it had never seen before, meaning it memorised examples instead of learning how addition actually works. As the problems got harder, most mistakes came from carry and borrow operations, and in mixed tasks, the model sometimes mixed up addition and subtraction. These errors followed clear patterns, showing that the model learned surface patterns but could not handle multi-step calculations.

On the other hand, the Boolean GPT learned faster and worked more reliably. It made fewer mistakes, mostly with XOR. Because Boolean logic uses a small and fixed set of rules, it matches well with the way Transformers learn patterns, which explains why the model performed better on Boolean tasks.

3.4 Implications and Limitations

Comparing Math GPT and Boolean GPT reveals an important limitation of Transformers. They work well when the task involves a small and fixed set of symbols, but they struggle with arithmetic problems that need step-by-step reasoning and tracking intermediate results. This issue still appears even when the model achieves high accuracy on restricted datasets.

There are ways this could be improved, such as training the model with step-by-step solutions or combining it with symbolic systems. However, these approaches were not explored in this project

Appendix A : Math GPT

This appendix presents representative examples from the **final Math GPT model**, trained on mixed two-digit addition and subtraction. The examples illustrate both correct behaviour and systematic failure modes.

A.1 Correct Predictions (Strengths)

The model is able to correctly solve many arithmetic expressions, particularly when no complex carry or borrow propagation is required.

```
47+38= → 85
82-32= → 50
22+66= → 88
67-58= → 9
```

These examples demonstrate that the model has learned meaningful arithmetic patterns and can apply the correct operation in many cases.

A.2 Incorrect Predictions (Weaknesses)

Despite reasonable overall performance, the model exhibits consistent and interpretable failure modes.

```
70+35= → 95      (expected 105)
28-11= → 6        (expected 17)
58+99= → 167      (expected 157)
66-28= → 39       (expected 38)
```

Most errors involve **carry or borrow propagation**, where local digit patterns are correct but global arithmetic consistency is violated. Occasional operator-related errors also appear, indicating incomplete binding between symbolic operators and their corresponding rules.

Appendix B : Boolean GPT

This appendix presents representative examples from the **final Boolean GPT model**, trained on Boolean logic expressions involving AND, OR, NOT, and XOR.

B.1 Correct Predictions (Strengths)

The model reliably evaluates common Boolean expressions, including unary and binary operators.

```
TrueANDFalse= → False
TrueORFalse=  → True
```

```
NOTTrue=          → False
(FalseORFalse)ANDTrue= → False
```

These examples show that the model has successfully learned core Boolean logic mappings.

B.2 Incorrect Predictions (Weaknesses)

A small number of errors were observed, primarily involving the XOR operator.

```
!python model/evaluate.py
```

```
Exact-match accuracy: 0.250 (1/4)
```

```
Sample errors:
```

```
FalseANDTrue= → predicted Fals, expected False
```

```
TrueXORTrue= → predicted True, expected False
```

```
TrueANDTrue= → predicted Fals. expected True
```

These errors suggest partial confusion between Boolean operators with overlapping truth-table structure, particularly XOR versus OR. Given the small dataset size, each individual error has a large impact on measured accuracy.